BARE METAL / SHARP X68000

PONG

C WITHOUT HUMAN68K ATTRACT-MODE DEMO WITH GAMEMODE

elf2x68k + IOCS + mode 12

Makefile -> dist/pong.xdf

DemoState keeps each mode responsibility explicit

Mode-specific state

TitleState
selected / idle_frames

DemoState
elapsed_frames

WinnerState
player / start

GameContext

application : ApplicationState

title : TitleState
pong : PongGameState
demo : DemoState
winner : WinnerState

Every state has a clear owner

make builds the bare-metal XDF end to end

01 /
COMPILE

m68k-xelf-gcc

-m68000 -O2
-specs=x68knodos.specs
Link address 0x6800

02 /
BOOT

Generate human.sys

Link directly as human.sys.
No Human68k system files
required.

03 / XDF

dist/pong.xdf

Create a new XDF containing
only human.sys.

512 x 480 display at 60 Hz

MODE

CRTMOD(12)

Standard 31 kHz mode:
512 x 512 / 65,536 colors.

SYNC

wait_vdisp()

Wait for V-DISP on MFP GPIP;
one update per frame.

TIMING

R05/R06/R07 -> R04

Set display position first;
set 525 scan lines last.

_ONTIME prevents lockups with a wait timeout

Add GAME_MODE_DEMO to the state machine

```
typedef enum {
  GAME_MODE_TITLE,
  GAME_MODE_PONG,
  GAME_MODE_DEMO,
  GAME_MODE_WINNER,
  GAME_MODE_EXIT
} GameModeId;

{demo_initialize,
 demo_update,
 demo_finalize}
```

Implement the three GameMode functions

01 initialize() starts two CPU players
02 update() checks input first
03 Run shared pong_game_update()
04 Return TITLE after 15 seconds
05 finalize() clears pending input

TITLE enters DEMO after 10 seconds without input.

demo_update() gives input the highest priority

```
static GameModeId demo_update(...) {
  if (wait_vdisp() != 0) return EXIT;
  if (input_has_activity()) return TITLE;

  pong_game_update(&context->pong, &input);
  if (++elapsed >= 60 * 15) return TITLE;
  return DEMO;
}
```

INPUT FIRST

Check input before the game update

SHARED STEP

Use the same PONG update as a normal match

10 SEC IDLE > DEMO > TITLE

Separate shared gameplay from mode transitions

01 /
TITLE

IDLE

Increment idle_frames each frame.
Enter DEMO after 10 seconds.

02 /
DEMO

PLAY

Assign CPU to both paddles and call
the shared pong_game_update().

03 /
RETURN

INPUT / TIME

Return to TITLE immediately on input
or after 15 seconds.

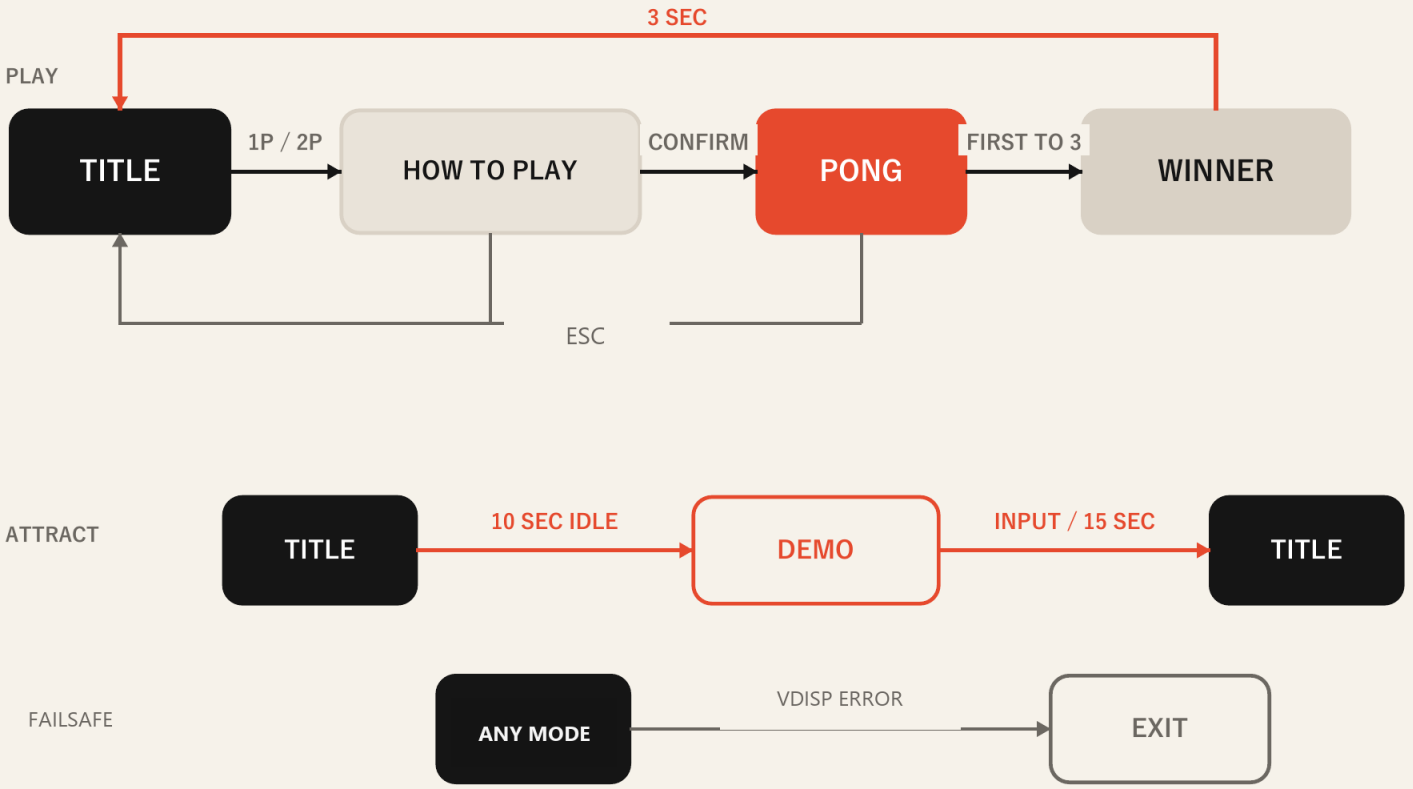
Add one GameMode; do not duplicate the game itself.

GameMode makes every screen transition explicit

GAME MODES

- **TITLE**
  Select 1P / 2P; track idle time
- **HOW_TO_PLAY**
  Explain controls; wait for release
- **PONG**
  Shared match; first to 3 points
- **WINNER**
  Show result; hold for 3 seconds
- **DEMO**
  CPU vs CPU; 15-second attract play
- **EXIT**
  Restore video mode; finish

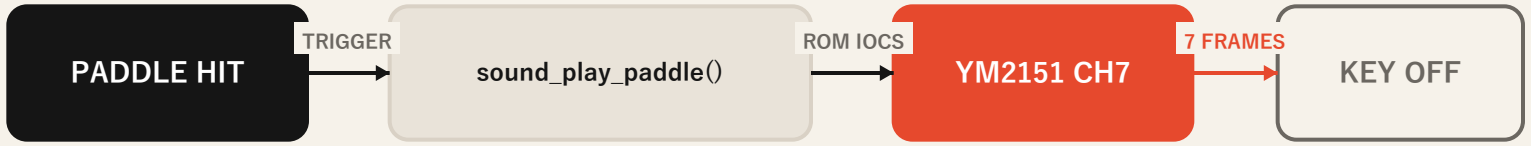
TRANSITION FLOW



A new screen adds one state and three lifecycle functions: initialize, update, finalize.

A seven-frame OPM chirp marks every paddle return

ONE EVENT, ONE CHANNEL



PITCH GLIDE · ONE STEP PER FRAME · ABOUT 0.12 SEC

COLLISION HOOK

```
if (ball_overlaps(...)) {
    pong_reflect(...);
    sound_play_paddle(sound);
}
```

MINIMAL STATE

```
SoundState
    int frames_left;
```

No queue. No allocator.

BARE-METAL PATH

- ROM IOCS _OPMSET ($68)
- YM2151 channel 7
- Algorithm 7 / four operators
- No timer IRQ or DMA
- Wall and score events stay silent

initialize: load patch **update: pitch + counter** **finalize: key off**